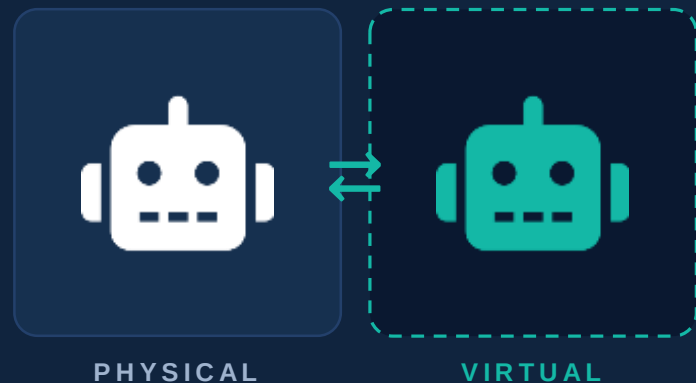


ROBOTICS TRAINEE PROGRAMME · THIS WEEK

Digital Twins for Robotics

Build a living virtual copy of a real machine — and learn why every serious robotics team on earth now tests in software before touching hardware.

Lecture pack · Intermediate track · Includes simulation + physical weekend practicals



live data · both ways



What Is a Digital Twin?

A digital twin is a virtual model of a physical object or system, continuously updated with real data from that object, and able to influence it in return.

Three ingredients — remove any one and it is no longer a twin:

1

The physical thing

A motor, a robot, a generator, a whole factory — something real that exists and behaves.

2

The virtual model

A software representation: geometry, physics, behaviour — detailed enough to answer real questions.

3

The live connection

Sensors stream reality into the model; decisions flow back. This link is what separates a twin from a mere simulation.



Model vs Shadow vs Twin: The Data-Flow Test

Industry abuses the word “twin.” Classify any system by asking: which way does the data flow automatically?

Digital MODEL

None — built by hand, updated by hand

A CAD file of your robot; a Gazebo world you edited manually

Digital SHADOW

Physical → virtual (one way, automatic)

A dashboard that live-plots your robot's battery and pose

Digital TWIN

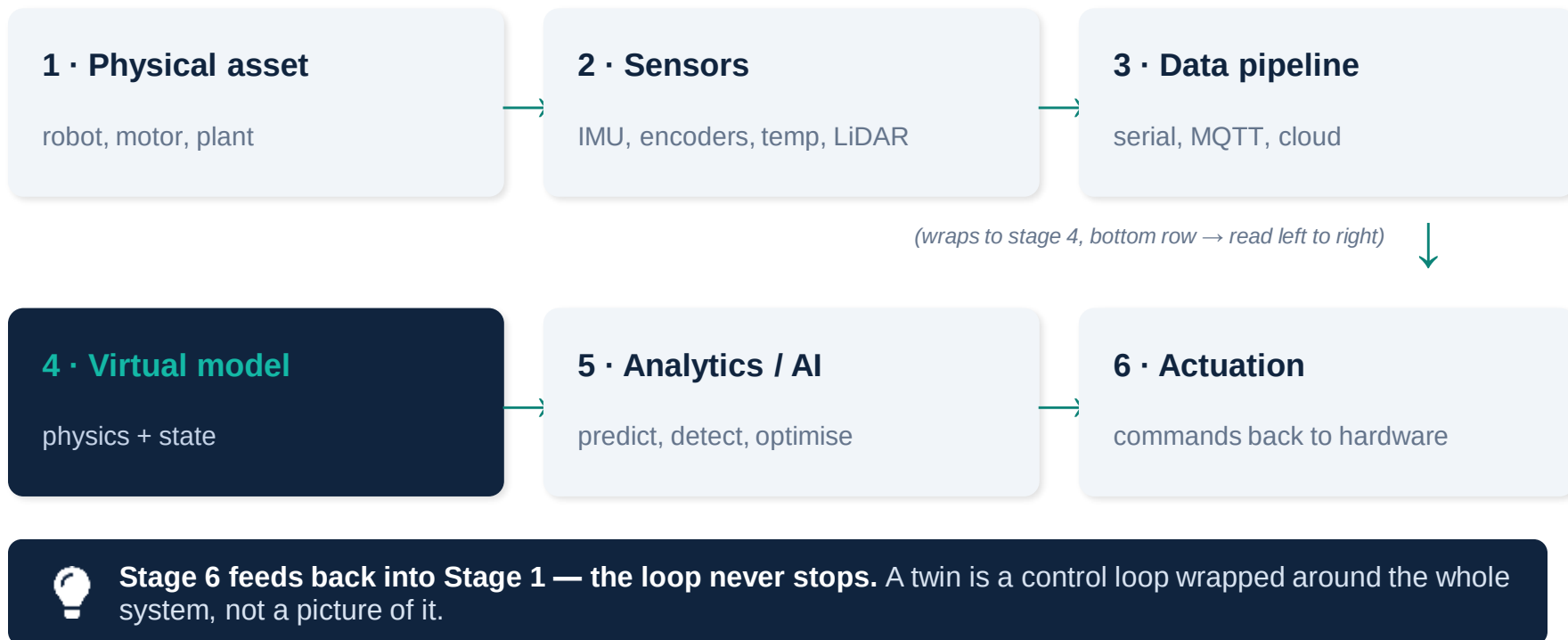
Physical ⇌ virtual (both ways, automatic)

The model detects motor overheating and commands the real robot to slow down

Exam question guaranteed: give one Nigerian example of each. Start thinking now.



Anatomy of a Twin: The Loop





How Deep Should the Twin Go? Fidelity Levels

Fidelity is a budget decision. Match the model depth to the question you need answered — no deeper.

L1 · Geometric

Shape and layout only

“Will the robot fit through that corridor?”

L2 · Kinematic

Motion without forces

“Where is the arm's gripper when joint 2 is at 40°?”

L3 · Physics / dynamic

Forces, friction, inertia, torque

“Will this payload tip the robot on a ramp?”

L4 · Behavioural + data-driven

Learned models, wear, failure statistics

“Which motor will fail first, and when?”

This week's practicals live at L1–L2. RAIN's drone work lives at L3. Predictive maintenance contracts live at L4.



Why Robotics Cannot Live Without Twins

Crash in software, not steel

A bad path in simulation costs nothing. The same bug on a physical robot costs a chassis, a motor driver — or a teammate's ankle.

Test 10,000× faster

Simulated robots run faster than real time and in parallel. One weekend of simulation = months of physical trials.

Train AI safely

Reinforcement learning needs millions of trials. You cannot do millions of trials on one physical robot — you can in its twin.

Predict before failure

A twin fed with vibration and temperature data flags a failing bearing weeks early. Downtime becomes a scheduled event.

Remember last week's rule: hardware is the last step, never the first.

Where Twins Are Working Today



Manufacturing

Factory lines tuned virtually before retooling; robots re-programmed offline while production keeps running.



Power & energy

Grid and turbine twins predict faults. Imagine a twin of a Nigerian feeder line flagging overloads before load-shedding.



Smart cities

Traffic twins test interventions virtually. A Lagos BRT-corridor twin could trial signal timings without touching a single junction.



Healthcare

Patient-specific heart and organ models let surgeons rehearse procedures before the first incision.



Agriculture

Farm twins fuse soil sensors, weather and drone imagery — irrigation and fertiliser decisions tested in software first.



Robotics & drones

Every serious lab flies the mission in the twin first. This is exactly how RAIN prototypes drone behaviour.



The Twin-Builder's Toolbox

Sense

Arduino / ESP32, IMUs, encoders, ultrasonic, DHT22, current sensors

the nervous system

Transport

Serial/USB for the bench; MQTT + Wi-Fi for the field; databases for history

the bloodstream

Model & simulate

Python (NumPy, Matplotlib), Webots, Gazebo, PyBullet, URDF robot descriptions

the imagination

Decide & display

Dashboards (Matplotlib live plots, Grafana, web apps), anomaly detection, ML models

the brain and face

Everything on this slide is already in the RAIN lab or free to install. No excuses.



The Sim-to-Real Gap: Where Twins Lie

A twin is only as honest as its calibration. Reality always disagrees with the model somewhere:

Friction & slip

Your simulated wheels never slip. Your real wheels slip on RAIN's tiled floor every single run.

Sensor noise

Simulated ultrasonic returns 42.0 cm. The real HC-SR04 returns 41, 44, 39, 213(!), 42...

Latency

In simulation, commands act instantly. In reality: serial delay + motor spin-up + battery sag.

Battery & wear

Motors weaken as voltage drops. A twin that ignores battery state drifts within minutes.



The professional habit — calibrate: measure the real system, feed measurements back into the model, repeat until twin and reality agree within tolerance. That loop IS the job.



Case Study: Twinning This Week's Grid Runner

Everything from the pathfinding lecture becomes a digital twin the moment we close the loop:

- 1 **The occupancy grid** is the twin's world model — a live map of the arena.
- 2 **The A* planner** is the twin's analytics layer — it reasons about the world model.
- 3 **The overhead photo** is the sensor feed — physical → virtual, updating the map.
- 4 **The command string (F, L, R)** is actuation — virtual → physical, closing the loop.
- 5 **Re-photograph + re-plan when a box moves** and congratulations: you have built a genuine two-way digital twin.

One project, two lectures, one loop. This is deliberate.



A Minimal Twin in 30 Lines

Arduino streams reality; Python mirrors it live. This skeleton is your weekend starting point.

Arduino — the physical side

```
// HC-SR04 -> serial stream
long readDistanceCm() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  return pulseIn(ECHO, HIGH) / 58;
}

void loop() {
  Serial.println(readDistanceCm());
  delay(100);    // 10 Hz
}
```

Python — the virtual side

```
import serial, collections
import matplotlib.pyplot as plt

ser = serial.Serial('COM5', 9600)
hist = collections.deque(maxlen=100)

plt.ion()
while True:
    d = float(ser.readline())
    hist.append(d)           # twin state
    plt.cla()
    plt.plot(hist); plt.ylim(0, 200)
    plt.title(f'Twin: {d:.0f} cm')
    plt.pause(0.01)
```

Add a rule — “if $d < 15$, send STOP back over serial” — and the shadow becomes a twin.



Exercise Ladder — Climb During the Week

1

Classify it **BEGINNER**

For 6 systems (Google Maps traffic, a CAD drawing, a CBT exam dashboard, a thermostat, a flight simulator, RAIN's LMS analytics), label each: model, shadow, or twin — and defend your answer in one sentence.

2

Spec a twin on paper **BEGINNER**

Pick any machine in the RAIN lab. List: 3 sensors you would attach, the data each produces, one decision the twin could automate. One page max.

3

Kinematic twin **INTERMEDIATE**

Write a Python class `DiffDriveTwin(x, y, theta)` that updates pose from wheel speeds (unicycle model). Animate it driving a square with `matplotlib`.

4

Noise modelling **INTERMEDIATE**

Log 200 real HC-SR04 readings of a fixed target. Plot the histogram, compute mean and std-dev, then make your simulated sensor produce the same distribution.

5

Predictive rule **ADVANCED**

Stream motor current while loading the shaft by hand. Design a rule (threshold or moving average) that flags “abnormal load” — your first predictive-maintenance twin.

Exercises 1–3 due before Friday's lab. 4–5 earn bonus marks.



Weekend Practical A — Simulation Mission

SOLO OR PAIRS

PYTHON + MATPLOTLIB (WEBOTS OPTIONAL)

SUBMIT: CODE + 60-SEC SCREEN
RECORDING

“The Ghost Robot” — a kinematic twin that replays your A mission*

- 1 Build DiffDriveTwin: state (x, y, θ) , method $\text{step}(v_{\text{left}}, v_{\text{right}}, dt)$ using the unicycle/differential-drive equations.
- 2 Feed it the F/L/R command string from your pathfinding weekend task. The twin should trace the same route through the same grid — draw both.
- 3 Inject realism: add Gaussian noise to wheel speeds ($\sigma = 5\%$). Run 20 trials and plot all 20 ghost trajectories over the ideal path.
- 4 Measure the drift: report mean final-position error across the 20 runs. This number is your sim-to-real gap, quantified.
- 5 Stretch: implement simple closed-loop correction (re-aim at the next waypoint each step) and show the error shrink.

If you have Webots installed: replicate steps 1–2 with the e-puck robot in a matching arena for extra credit.



Weekend Practical B — Physical Build

TEAMS OF 3

ARDUINO + HC-SR04 + SERVO (LAB KITS)

LAPTOP RUNNING THE PYTHON TWIN

“The Room Twin” — a live radar map of a real corner of the lab

- 1 Mount the HC-SR04 on a servo. Sweep 0–180° in 5° steps; at each angle print “angle,distance” over serial.
- 2 Python side: read the stream and convert to Cartesian points — $x = d \cdot \cos(\theta)$, $y = d \cdot \sin(\theta)$. Plot a live polar/scatter map.
- 3 Place 2–3 boxes in front of the rig. Your twin's map should show them as point clusters. Move a box — the map must follow within one sweep.
- 4 Close the loop: if any object enters the 20 cm “keep-out zone,” Python sends 'A' back over serial and the Arduino sounds a buzzer / LED.
- 5 Calibrate honestly: measure one box with a tape rule vs your map. Report the error in cm — every good twin ships with an accuracy figure.

Deliverable: 2-minute video — rig, live map reacting to a moving box, buzzer firing. Demo day: Monday 9 AM.



Key Takeaways

-  A digital twin = physical asset + virtual model + a live two-way data link. All three, always.
-  Twins let robotics teams crash in software, test thousands of times faster, and train AI safely.
-  The data-flow test separates model (no auto flow), shadow (one way) and twin (both ways).
-  The sim-to-real gap is permanent — calibration against real measurements is the core professional skill.
-  Fidelity is a budget: model only as deep as the question you need answered.
-  Your weekend projects ARE twins: the sweep map mirrors reality; the STOP command closes the loop.

Next week we join the two threads: the Grid Runner drives inside its own twin, and the twin re-plans with A when the world changes.*